Project Update: Dynamics and Stability of the Spring-Loaded Inverted Pendulum (SLIP) Model

Arael Anaya

(a) System and Motivation

This project models the spring-loaded inverted pendulum (SLIP), the reduced-order model behind most legged-robot dynamics [3, 5]: a point mass bouncing on a massless spring leg. It captures real running dynamics well [1, 2] while staying simple enough to analyze with this course's tools. The stance phase is an inverted spring pendulum, extending the elastic pendulum from class.

=====

(b) Modeling Methods

The SLIP model is hybrid (Figure 2): a spring-loaded pendulum in stance, a projectile in flight. Each phase's equations of motion come from the Lagrangian, in polar coordinates (r, θ) for stance, plus the touchdown/liftoff switching conditions connecting them. Gait stability is assessed with an apex return map – a Poincaré map where a periodic hop is a fixed point [1, 4]. The model runs in Python with event-based integration, sweeping spring stiffness and touchdown angle.

(c) Progress to Date

Both phases' equations of motion and the touchdown map are derived below and implemented in `main.py` as `touchdownState()` and `stance_RHS()`. Figure 2 shows the geometry.

Flight-Phase Equations of Motion

With basis $\hat{n}_1, \hat{n}_2$, $\vec{r}_m = x\hat{n}_1 + y\hat{n}_2$:

$$T = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2), \qquad V = mgy, \qquad L = T - V = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2) - mgy$$

Lagrange's equations give

$$\ddot{x} = 0, \qquad \ddot{y} = -g$$

Starting at the apex ($\dot{y} = 0$), with $x(0) = x_0$, $\dot{x}(0) = v_{x0}$:

$$x(t) = v_{x0}t + x_0, \qquad y(t) = y_0 - \frac{g}{2}t^2$$

Touchdown: Detection and Transition Map

θ_{TD} is held fixed in flight, so the foot trails the body at

$$\vec{r}_{foot}(t) = \big(x(t) - r_0\cos\theta_{TD}\big)\hat{n}_1 + \big(y(t) - r_0\sin\theta_{TD}\big)\hat{n}_2$$

Touchdown is $y_{foot} = 0$, i.e. $y_{TD} = r_0\sin\theta_{TD}$. Solving $y(t) = y_0 - \frac{g}{2}t^2$:

$$t_{TD} = \sqrt{\frac{2(y_0 - r_0\sin\theta_{TD})}{g}}, \qquad \dot{y}_{TD} = -\sqrt{2g(y_0 - r_0\sin\theta_{TD})}, \qquad x_{TD} = v_{x0}t_{TD}, \qquad \dot{x}_{TD} = v_{x0}$$

Stance uses polar coordinates about the planted foot, so this Cartesian state must convert. With foot at $\vec{r}_F = (x_{TD} + r_0\cos\theta_{TD})\hat{n}_1$ and leg vector $\vec{\rho} = \vec{r}_m - \vec{r}_F = -r_0\cos\theta_{TD}\,\hat{n}_1 + y_{TD}\hat{n}_2$:

$$r = |\vec{\rho}| = \sqrt{r_0^2\cos^2\theta_{TD} + y_{TD}^2} = r_0, \qquad \theta = \operatorname{atan2}\big(y_{TD},\, -r_0\cos\theta_{TD}\big)$$

Projecting $\vec{v} = \dot{x}_{TD}\hat{n}_1 + \dot{y}_{TD}\hat{n}_2$ onto $\hat{e}_r = \cos\theta\,\hat{n}_1 + \sin\theta\,\hat{n}_2$, $\hat{e}_\theta = -\sin\theta\,\hat{n}_1 + \cos\theta\,\hat{n}_2$:

$$\dot{r} = \vec{v}\cdot\hat{e}_r = \dot{x}_{TD}\cos\theta + \dot{y}_{TD}\sin\theta, \qquad \dot{\theta} = \frac{\vec{v}\cdot\hat{e}_\theta}{r_0} = \frac{-\dot{x}_{TD}\sin\theta + \dot{y}_{TD}\cos\theta}{r_0}$$

implemented as `touchdownState()`.

Stance-Phase Equations of Motion

In stance the leg pivots about the fixed foot: $\vec{r}_m = r\hat{e}_r$, $\vec{v}_m = \dot{r}\hat{e}_r + r\dot{\theta}\hat{e}_\theta$:

$$T = \frac{1}{2}m\dot{r}^2 + \frac{1}{2}mr^2\dot{\theta}^2, \qquad V = mgr\sin\theta + \frac{1}{2}k(r-r_0)^2, \qquad L = T - V$$

Lagrange's equations:

$$m\ddot{r} - mr\dot{\theta}^2 + mg\sin\theta + k(r-r_0) = 0, \qquad mr^2\ddot{\theta} + 2mr\dot{r}\dot{\theta} + mgr\cos\theta = 0$$

solved for the highest derivatives:

$$\ddot{r} = r\dot{\theta}^2 - g\sin\theta - \frac{k}{m}(r-r_0), \qquad \ddot{\theta} = -\frac{2}{r}\dot{r}\dot{\theta} - \frac{g}{r}\cos\theta$$

implemented as `stance_RHS()`.

Conserved Energy in Stance

$\vec{r}_m = r\hat{e}_r$ is scleronomic and T is homogeneous quadratic in $(\dot{r}, \dot{\theta})$, so $H = T + V$; with no explicit t-dependence, energy is conserved through stance:

$$E = \frac{1}{2}m(\dot{r}^2 + r^2\dot{\theta}^2) + mgr\sin\theta + \frac{1}{2}k(r-r_0)^2 = \text{const.}$$

Neither coordinate is cyclic, so both equations must be integrated together; this conserved energy checks the integrator.

Numerical Results

Parameters:

m	r_0	k	g	θ_{TD}	y_0	v_{x0}
80 kg	1 m	20000 N/m	9.81 m/s^2	68°	1.2 m	3.0 m/s

`main.py` maps the apex state to stance via `touchdownState()`, integrates `stance_RHS()` with `solve_ivp` (`rtol` $= 10^{-10}$, `atol` $= 10^{-12}$) to the liftoff event $r = r_0,\ \dot{r} > 0$, checking energy at every step:

	r [m]	θ [deg]	$\dot{r}$ [m/s]	$\dot{\theta}$ [rad/s]
Touchdown	1.0000	112.00	−3.2689	−1.9149
Liftoff	1.0000	83.48	3.1937	−1.6886

Stance lasts 0.2094 s; the leg compresses to 0.7740 m (22.6% of r_0); relative energy drift is 6.46×10^{-13}, i.e. machine precision. Figure 1 plots leg length, angle, CoM trajectory, and energy drift.

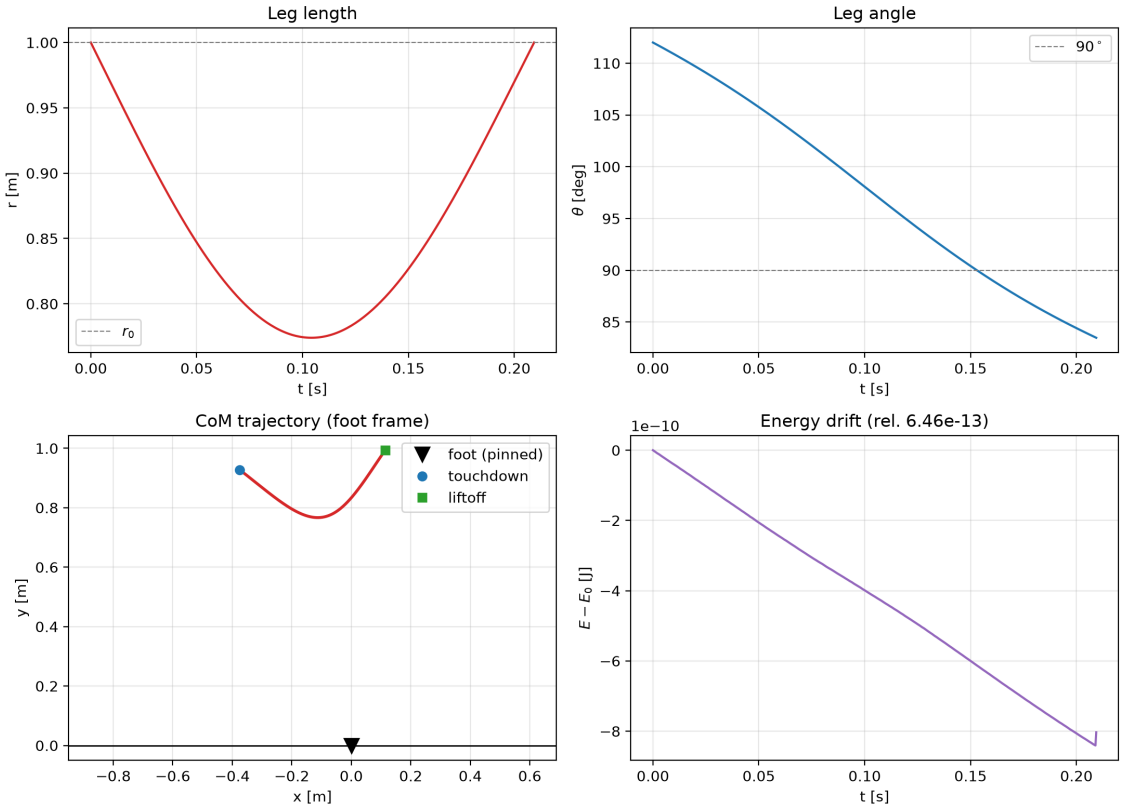

Figure 1: Stance simulation: $r(t)$, $\theta(t)$ (top), CoM trajectory (bottom left), energy drift (bottom right).

Remaining Work

This single-stance simulation is validated end to end. Remaining: map liftoff back to flight variables, chain hops into the apex return map from part (b) [6], then sweep (k, θ_{TD}) for stable gaits.

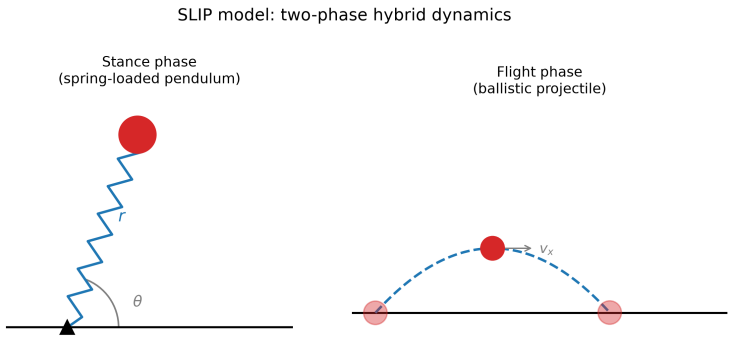

Figure 2: SLIP model geometry: stance (spring pendulum, r, θ) and flight (projectile).

Scheduled update meeting: Friday, August 7, 2026, 10:30–11:00 AM, with Gary Nave (remote).

References

[1] R. Blickhan, "The spring-mass model for running and hopping," *Journal of Biomechanics*, vol. 22, no. 11–12, pp. 1217–1227, 1989.

[2] T. A. McMahon and G. C. Cheng, "The mechanics of running: how does stiffness couple with speed?," *Journal of Biomechanics*, vol. 23, suppl. 1, pp. 65–78, 1990.

[3] R. J. Full and D. E. Koditschek, "Templates and anchors: neuromechanical hypotheses of legged locomotion on land," *Journal of Experimental Biology*, vol. 202, no. 23, pp. 3325–3332, 1999.

[4] W. J. Schwind and D. E. Koditschek, "Approximating the stance map of a 2-DOF monoped runner," *Journal of Nonlinear Science*, vol. 10, pp. 533–568, 2000.

[5] U. Saranli, M. Buehler, and D. E. Koditschek, "RHex: a simple and highly mobile hexapod robot," *International Journal of Robotics Research*, vol. 20, no. 7, pp. 616–631, 2001.

[6] H. Geyer, A. Seyfarth, and R. Blickhan, "Compliant leg behaviour explains basic dynamics of walking and running," *Proceedings of the Royal Society B*, vol. 273, no. 1603, pp. 2861–2867, 2006.

Appendix: Code

`main.py` – touchdown map, stance-phase RHS, and the single-stance simulation and validation plots discussed in (c).

```python
import numpy as np
from scipy.integrate import solve_ivp

# Paramerts
m = 80
r_o = 1
k = 20000
g = 9.81
theta_TD = np.radians(68)
y_o = 1.2
v_o = 3.0

params = {'m' : m , 'k' : k , 'g' : g, 'r_o' : r_o}


def touchdownState(y_o , v_o, theta_TD, r_o, g):
    r = r_o
    theta = np.arctan2(r_o * np.sin(theta_TD) , -r_o * np.cos(theta_TD))

    x_TD_dot = v_o
    y_TD_dot = - (2*g*(y_o - r_o * np.sin(theta_TD)))**.5


    r_dot = x_TD_dot * np.cos(theta) + y_TD_dot * np.sin(theta)

    theta_dot = (-x_TD_dot * np.sin(theta) + y_TD_dot * np.cos(theta)) / r_o

    return [r , theta, r_dot , theta_dot]

def liftoff(t, s, params):
    return params['k'] * (s[0] - params['r_o'])
liftoff.terminal = True
liftoff.direction = 1


def stance_RHS(t , s, params):
    r_dot = s[2]
    theta_dot = s[3]

    r_ddot = s[0] * theta_dot**2 - params['g'] * np.sin(s[1]) - (params['k'] / params['m'
        ])* (s[0] - params['r_o'])
    theta_ddot = -(2/s[0]) * r_dot * theta_dot - (params['g'] / s[0]) * np.cos(s[1])

    return [r_dot , theta_dot, r_ddot, theta_ddot]
```

```
44
45 def energy(s, params):
46     E = (1/2) * params['m'] * (s[2]**2 + s[0]**2 * s[3]**2) + params['m'] * params['g'] *
           s[0] * np.sin(s[1]) + (1/2) * params['k']* (s[0] - params['r_o'])**2
47
48     return E
49
50 s0 = touchdownState(y_o , v_o, theta_TD, r_o, g)
51 t_span = (0.0,1.0)
52
53 sol = solve_ivp(stance_RHS, t_span, s0, args=(params,),
54                 events=liftoff, rtol=1e-10, atol=1e-12,
55                 dense_output=True, max_step=1e-3)
56
57 E = np.array([energy(sol.y[:, i], params) for i in range(sol.y.shape[1])])
58 drift = np.max(np.abs(E - E[0])) / np.abs(E[0])
59
60 print("status:", sol.status)
61 print("liftoff time:", sol.t_events[0])
62 print("drift:", drift)
63
64 import matplotlib.pyplot as plt
65
66 # ---------- diagnostics ----------
67 r_TD, th_TD, rdot_TD, thdot_TD = s0
68 s_LO = sol.y_events[0][0]
69 t_LO = sol.t_events[0][0]
70
71 print("--- touchdown state ---")
72 print(f"  r       = {r_TD:.6f} m")
73 print(f"  theta   = {np.degrees(th_TD):.4f} deg")
74 print(f"  r_dot   = {rdot_TD:.6f} m/s")
75 print(f"  th_dot  = {thdot_TD:.6f} rad/s")
76
77 print("--- liftoff state ---")
78 print(f"  status         = {sol.status}")
79 print(f"  stance duration= {t_LO:.6f} s")
80 print(f"  r              = {s_LO[0]:.6f} m")
81 print(f"  theta          = {np.degrees(s_LO[1]):.4f} deg")
82 print(f"  r_dot          = {s_LO[2]:.6f} m/s")
83 print(f"  th_dot         = {s_LO[3]:.6f} rad/s")
84
85 print("--- validation ---")
86 print(f"  min leg length = {np.min(sol.y[0]):.6f} m")
87 print(f"  max compression= {params['r_o'] - np.min(sol.y[0]):.6f} m")
88 print(f"  E_0            = {E[0]:.6f} J")
89 print(f"  relative drift = {drift:.3e}")
90
91 # ---------- figures ----------
92 t = sol.t
93 r = sol.y[0]
94 th = sol.y[1]
95 x_com = r * np.cos(th)
96 y_com = r * np.sin(th)
97
98 fig, ax = plt.subplots(2, 2, figsize=(11, 8))
99
100 ax[0, 0].plot(t, r, color='C3')
101 ax[0, 0].axhline(params['r_o'], ls='--', lw=0.8, color='gray', label='$r_0$')
102 ax[0, 0].set_xlabel('t [s]')
103 ax[0, 0].set_ylabel('r [m]')
104 ax[0, 0].set_title('Leg length')
```

```
ax[0, 0].legend()

ax[0, 1].plot(t, np.degrees(th), color='C0')
ax[0, 1].axhline(90, ls='--', lw=0.8, color='gray', label='$90^\\circ$')
ax[0, 1].set_xlabel('t [s]')
ax[0, 1].set_ylabel(r'$\theta$ [deg]')
ax[0, 1].set_title('Leg angle')
ax[0, 1].legend()

ax[1, 0].plot(x_com, y_com, color='C3', lw=2)
ax[1, 0].plot(0, 0, 'kv', ms=10, label='foot (pinned)')
ax[1, 0].plot(x_com[0], y_com[0], 'o', color='C0', label='touchdown')
ax[1, 0].plot(x_com[-1], y_com[-1], 's', color='C2', label='liftoff')
ax[1, 0].axhline(0, color='k', lw=1)
ax[1, 0].set_xlabel('x [m]')
ax[1, 0].set_ylabel('y [m]')
ax[1, 0].set_title('CoM trajectory (foot frame)')
ax[1, 0].axis('equal')
ax[1, 0].legend()

ax[1, 1].plot(t, E - E[0], color='C4')
ax[1, 1].set_xlabel('t [s]')
ax[1, 1].set_ylabel('$E - E_0$ [J]')
ax[1, 1].set_title(f'Energy drift (rel. {drift:.2e})')
ax[1, 1].ticklabel_format(axis='y', style='sci', scilimits=(0, 0))

for a in ax.flat:
    a.grid(alpha=0.3)
fig.tight_layout()
fig.savefig('stance_validation.png', dpi=200)
plt.show()
```